PlayZone: Sports Venue Discovery, Booking & Tournament Management Platform

A project report submitted in the partial fulfilment of the award of degree of

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING

By

M PRADEEP	**231801390016**
AS VIJAYKUMAR	**231801390031**
CH ARUNKUMAR	**231801390032**
Y DINESH	**231801390029**
B ANILKUMAR	**231801390020**
D PAVANESWAR	**231801390017**
DN KAILASH	**231801390007**

Under the esteemed Guidance of

Dr. N.V.S. SHANKAR
Assistant Professor



Centurion
UNIVERSITY

Centurion University of Technology and Management
Andhra Pradesh, Pin: 535003, India (2025-2026)

BONAFIDE CERTIFICATE

Certified that this project report **PLAYZONE: SPORTS VENUE DISCOVERY, BOOKING & TOURNAMENT MANAGEMENT PLATFORM** is the Bonafide work of "**M PRADEEP(231801390016), AS VIJAYKUMAR(231801390031), CH ARUNKUMAR(231801390032), Y DINESH(231801390029), B ANILKUMAR(231801390020), D PAVANESWAR(231801390017), DN KAILASH(231801390007)**" who carried out the project work under my supervision. This is to further certify to the best of my knowledge that this project has not been carried out earlier in this institute and the university.

SIGNATURE
Dr. N. V. S. SHANKAR
Assistant Professor

Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university.

SIGNATURE
R. LAKSHMANA RAO
Assistant Professor, HOD (CSE)

SIGNATURE
Dr. Satyabrata Lenka
Professor & DEAN, SoET

HEAD OF THE DEPARTMENT / DEAN OF THE SCHOOL

Professor of Computer Science and Engineering
DEPARTMENT SEAL

DECLARATION

We declare that this project report titled **PLAYZONE: SPORTS VENUE DISCOVERY, BOOKING & TOURNAMENT MANAGEMENT PLATFORM** submitted in partial fulfillment of the degree of **B.Tech in Computer Science and Engineering** is a record of original work carried out by us under the supervision of **Dr. N.V.S. SHANKAR**, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgments have been made wherever the findings of others have been cited.

M PRADEEP	**231801390016**
AS VIJAYKUMAR	**231801390031**
CH ARUNKUMAR	**231801390032**
Y DINESH	**231801390029**
B ANILKUMAR	**231801390020**
D PAVANESWAR	**231801390017**
DN KAILASH	**231801390007**

INDEX

Contents

PlayZone: Sports Venue Discovery, Booking & Tournament Management Platform	**1**
1. INTRODUCTION	3
2. SYSTEM OVERVIEW	4
3. ARCHITECTURE DESIGN	5
4. COMPONENT DESIGN	6
5. DATA DESIGN	7
6. DATA FLOW AND RUNTIME WORKFLOWS	7
7. DEPLOYMENT AND CONFIGURATION	8
8. LIMITATIONS AND FUTURE ENHANCEMENTS	9
9. CONCLUSION	9

ACKNOWLEDGEMENT

We are immensely thankful to Associate Professor **Dr. N.V.S.SHANKAR** of the Department of Computer Science and Engineering at SoET, Vizianagaram Campus. He led us through the complexities of this project effortlessly, displaying unparalleled generosity and guidance.

We thank Asst. Prof. **R. Lakshmana Rao**, Head of the Department of Computer Science and Engineering, SoET, Vizianagaram Campus for extending their support during the course of this investigation.

We thank **Dr. D. Srinivasa Rao**, Dean of SoET, Vizianagaram Campus for their invaluable guidance, insightful feedback, and continuous support throughout the course of this project. Your expertise and mentorship have been invaluable.

We thank **Dr. P. Pallavi**, Registrar, CUTM, Vizianagaram Campus for their assistance and cooperation in facilitating the necessary resources and administrative support essential for the successful execution of this project.

We thank **P. Prasanta Kumar Mohanty**, Vice Chancellor, CUTM, Vizianagaram Campus for fostering an environment that encourages academic excellence and innovation. Your vision has been a constant source of inspiration.

We also express our deepest appreciation to our parents for their unconditional love, encouragement, and belief in our abilities. Their unwavering support has been the cornerstone of our achievements.

We are sincerely grateful to each one of you for your contributions, guidance, and unwavering support, without which this project would not have been possible.

Software Requirement Specifications

1. Scope

The PlayZone: Sports Venue Discovery, Booking & Tournament Management Platform is a modern, native Android solution designed to streamline athletic facility operations and elevate community sports engagement. The platform bridges the operational divide between Venue Managers—who administer arenas, turfs, sports courts, equipment inventories, operating schedules, and match scoreboards—and Sports Players, who discover facilities, book session slots, track booking approvals, and monitor real-time game scores. By incorporating rigorous transactional conflict detection, biometric device verification, and offline-first Room persistence with Google Sheets cloud backup, the platform delivers high transparency, operational resilience, and professional sports management across academic and municipal environments.

1.1 Identification

Identification No	SRS-PZ-VER-1.0
Title	PlayZone: Sports Venue Discovery, Booking & Tournament Management Platform
Abbreviation	PZ-SVTM
Version No	1.0
Release No	1.0

1.2 System Overview

PlayZone (PZ-SVTM) provides a unified mobile experience engineered with Kotlin and Jetpack Compose on Android. The architecture integrates a robust local Room Database (v3) with fallback migration, Google Sheets Apps Script API and published CSV endpoints for lightweight cloud synchronization, and Firebase Authentication for enterprise credential issuance. The platform delivers sport-specific live scoring interfaces (Cricket, Football, Basketball, Badminton, Volleyball, Tennis), automated schedule conflict prevention during booking approvals, WorkManager background reminder dispatches, and PDF standings generation for tournament organisers and university athletics boards.

1.3 Document Overview

This Software Requirements Specification document outlines the functional capabilities, technical constraints, security standards, external interfaces, and database schema governing PlayZone. It serves as the baseline contract for system evaluation, verification, maintenance, and deployment across university campuses and athletic clubs.

1.4 Software and Hardware Requirements

The PlayZone application operates on Android devices executing Android 8.0 (API Level 26) or later, with a minimum SDK requirement of API 24 (Android 7.0 Nougat) and target SDK 36. The frontend is built using Jetpack Compose with Material 3 design tokens, integrated with Room persistence (v2.8.5), Kotlin Coroutines (1.10.2), WorkManager (2.12.0), Android BiometricPrompt (1.2.0), and Firebase Authentication. Development requires Android Studio (Ladybug / Quail), JDK 17+, an Intel Core i5 processor (or equivalent), 8 GB RAM minimum (16 GB recommended), and 256 GB SSD. Client smartphones require at least 2 GB RAM, a capacitive touch display, biometric hardware (or screen PIN lock), and internet connectivity for cloud synchronization.

1.5 Functional Requirements

i. Data Management

The platform persists users, sports arenas, equipment inventories, match sessions, player bookings, system notifications, verified reviews, match scoreboards, and tournament brackets within a local Room database ('ps22.db'). A bi-directional Google Sheets synchronization pipeline connects published CSV rosters and Google Apps Script endpoints to ensure reliable data recovery, offline usability, and administrative reporting.

ii. Administration

Venue Managers access a dedicated operational dashboard to register sports arenas, manage equipment pricing, define daily time slots, monitor confirmed reservations, inspect live match scoreboards, and review pending booking requests with transactional conflict protection that prevents double-booking across overlapping session times.

iii. User Interaction

Sports Players discover nearby arenas through categorized sports chips and search filters, review amenities, select available time slots, request equipment rentals, track booking approvals via real-time notification badges, monitor sport-specific live scoreboards, and export tournament standings directly to printable PDF documents.

1.5.1 Configuration

The application is compiled using Android Gradle Plugin 8.9+, Kotlin 2.1+, and Compose Compiler plugin. It configures the application ID 'com.ps22.tournament', minSdk 24, compileSdk 36, targetSdk 36, and integrates Google Play Services Auth with strict fallback to on-device authenticated accounts.

1.6 Development Details

S. No	Role	Details
1	Developer-Admin	Centurion University of Technology and Management (SoET CSE)
2	User-Client	Venue Managers, Tournament Organisers, Student Athletes, Faculty

3. Requirements

3.1 Required Mode of Operation

PlayZone operates in dual modes: online cloud-connected mode and fully autonomous offline-first mode. In offline mode, all user interactions, venue discoveries, local slot reservations, and live scoreboard adjustments persist immediately into the local Room database. Upon network restoration, background workers push pending synchronizations to Google Sheets and refresh published schedules. Role-based security routes users seamlessly based on their persistent role entity.

3.2 System Capability Requirements

3.2.1 System Software Requirements

The system software utilizes Jetpack Compose BOM 2026.04.01 with Material 3 UI, Room KTX with Room Compiler (2.8.5), WorkManager KTX (2.12.0) for background notifications, AndroidX Credentials (1.3.0), Google Identity Libraries, and AndroidX Biometric (1.2.0-alpha05) for high-assurance local device authentication.

3.2.2 Application Software Requirements

The application enables Venue Managers to administer sports facilities, create sessions, manage rental gear, approve or reject booking requests with conflict validation, and control live match scoreboards. Athletes discover arenas, book time slots, track reservations, receive alerts, and export PDF standings.

3.3 Requirements Table

Table 1: Software Requirements

ID No	Software Requirements
PZ-01	Branded Landing & Google Sign-In Authentication
PZ-02	Role Selection & Profile Onboarding (Venue Manager vs Player)
PZ-03	Sports Venue Discovery, Filtering & Categorization
PZ-04	Venue Details, Amenities Breakdown & Equipment Inventory
PZ-05	Interactive Time Slot Selection & Booking Dispatch
PZ-06	Manager Schedule Grid & Transactional Conflict Detection
PZ-07	Booking Requests Approval & Reason-Based Rejection Flow
PZ-08	Multi-Sport Live Match Scoreboard (Cricket, Football, Basketball, etc.)
PZ-09	WorkManager Session Reminder Alerts & Notification Feed
PZ-10	Tournament Bracket Engine & Standings PDF Report Export

3.4 Detailed Description of Requirements

ID	Description
PZ-01	**Feature: Google Sign-In & Identity Authentication** **Input:** User initiates 'Continue with Google' or selects saved device account. **Process:** • Invoke Android Credential Manager / Google ID Token Option. • Validate user email and token signature. • Enforce local device verification (BiometricPrompt / screen lock). • Query Room Database for existing role assignment. **Output:** Authentication confirmed; routes to Role Selection or Main Workspace.
PZ-02	**Feature: Role Selection & Profile Onboarding** **Input:** User selects 'Venue Manager' or 'Sports Player' card; optional phone. **Process:** • Capture selected role and persist into UserEntity table. • Initialize personalized seed venues, equipment, and match sessions. • Configure role-specific bottom navigation bar. **Output:** User profile saved; tailored workspace loaded.
PZ-03	**Feature: Sports Venue Discovery & Search** **Input:** Sport category chip selection (Football, Cricket, Badminton, etc.), search queries. **Process:** • Filter active VenueEntity records by supported sport, city, or area. • Sort venues by verified customer star ratings and proximity. • Display hero cards with sport photography, facility chips, and pricing. **Output:** Filtered list of verified venues displayed dynamically.

ID	Description
PZ-04	**Feature: Interactive Session Booking & Request Dispatch** **Input:** Date, time slot selection, requested equipment, player phone. **Process:** • Validate session status is 'AVAILABLE' in SessionEntity. • Calculate total booking fee including equipment add-ons. • Generate unique reference (e.g. BK-20261012-0042). • Insert BookingEntity with status 'PENDING'; dispatch manager notification. **Output:** Booking request submitted; reference shown in Player Bookings.
PZ-05	**Feature: Venue Manager Schedule & Conflict Protection** **Input:** Booking request review; Manager taps 'Approve' or 'Reject'. **Process:** • Transactional check for overlapping CONFIRMED bookings on the same venue/slot. • If overlap detected, prevent approval and alert manager to conflict. • On approval, update session to 'BOOKED', create MatchScoreboardEntity. • On rejection, record formal reason ('Venue maintenance', 'Time unavailable'). **Output:** Session state updated; confirmation notification sent to Player.
PZ-06	**Feature: Multi-Sport Live Match Scoreboard** **Input:** Real-time score increments, wickets, overs, goal events, point changes. **Process:** • Format scoreboard display tailored to sport (Cricket runs/overs, Football timer/goals). • Update MatchScoreboardEntity and append timeline event to ScoreboardEventEntity. • Emit reactive StateFlow update to all connected spectator views. **Output:** Live match dashboard reflects instantaneous game progress.
PZ-07	**Feature: Google Sheets Cloud Synchronization & PDF Export** **Input:** Automated background trigger or manual export request. **Process:** • WorkManager queries pending sync queue and dispatches payloads to Google Apps Script. • StandingsPdf generator compiles league rankings, goal differentials, and match results. • Writes standard PDF document using Android PdfDocument canvas API. **Output:** Cloud sheet synced; formatted PDF exported to external storage.

3.5 CSCI (Computer Software Configuration Item)

The PlayZone system consists of a Kotlin-based Android application (com.ps22.tournament) executing on Jetpack Compose, a Room local relational database for persistent state, a Google Sheets Apps Script API for cloud backup, and Firebase Authentication for secure identity tokens. The business logic incorporates an automated slot conflict detection engine, a sport-aware live scoring engine, a WorkManager scheduling subsystem, and an embedded PDF rendering engine.

3.6 CSCI External Interface Requirements

PlayZone integrates with Google Play Services Auth and Credential Manager for OAuth 2.0 token acquisition, Firebase Authentication for cloud role verification, Google Sheets Apps Script endpoints via HTTP REST, Android BiometricPrompt for local hardware authentication, and Android FileProvider for sharing exported tournament PDF standings.

3.7 CSCI Internal Interface Requirements

Internal components communicate through clean architectural boundaries: Compose UI screens observe reactive StateFlow streams exposed by ViewModels and repositories. The repositories coordinate local Room DAOs (VenueBookingDao, TournamentDao) and background workers (ReminderScheduler) to ensure consistent transaction lifecycles.

3.8 CSCI Internal Data Requirements

All critical records—including user profiles, arena listings, rental equipment inventory, match sessions, reservations, live scoreboards, and notifications—are stored within Room Database version 3 ('ps22.db'). Relational integrity is enforced through foreign key indices, unique booking references, and transactional queries.

3.9 Adaptation Requirements

The system accommodates diverse institutional athletic facilities (turf football grounds, indoor badminton courts, cricket stadiums, swimming complexes) with configurable operating hours, custom pricing rules, equipment add-ons, and dynamic sport scoring models.

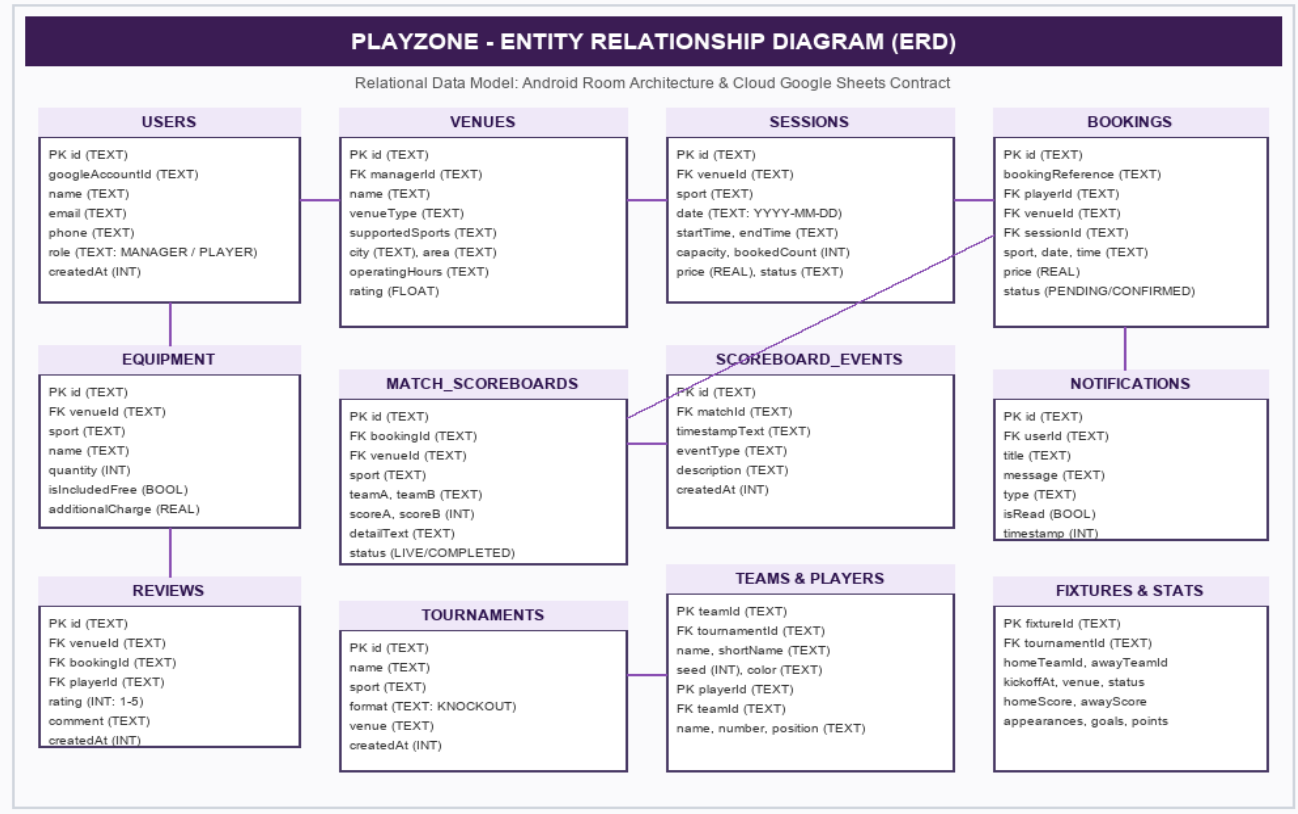
3.10 Security and Privacy Requirements

Role-based authorization guarantees that only verified Venue Managers can modify arena settings and approve booking requests. Device-level authentication (BiometricPrompt / system lock) protects user accounts on shared devices. Cloud communication operates exclusively over encrypted TLS/HTTPS channels.

4. Database backend

PlayZone: Sports Venue Discovery, Booking & Tournament Management Platform Database Schema

4.1 Entity Relationship Diagram (ERD)



The Entity Relationship Diagram (ERD) represents the relational database structure powering PlayZone. The schema originates with Users (Venue Managers and Sports Players). Venue Managers operate Venues, which maintain Equipment inventories and publish daily Sessions. Players create Bookings against available sessions, generating unique booking references. When a booking is confirmed, the system initializes a MatchScoreboard with sport-tailored scoring (Cricket, Football, Basketball, Badminton), while ScoreboardEvents track chronological match moments. Concurrently, Notifications and Reviews link players and venues to maintain high accountability and service quality. Legacy Tournament, Team, Player, and Fixture entities provide full compatibility for organized leagues.

Centurion University of Technology and Management

Department of Computer Science and Engineering

SOFTWARE DESIGN DOCUMENT

PlayZone

Sports Venue Discovery, Booking & Tournament Management Platform

M PRADEEP	231801390016
AS VIJAYKUMAR	231801390031
CH ARUNKUMAR	231801390032
Y DINESH	231801390029
B ANILKUMAR	231801390020
D PAVANESWAR	231801390017
DN KAILASH	231801390007

Under the guidance of

Dr. N V S Shankar, CSE Department

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Design Document (SDD) is to detail the architectural framework, component hierarchy, database structure, reactive state pipelines, security protocols, and operational workflows of PlayZone. It guides engineering maintainers, quality assurance auditors, and academic evaluators in understanding how the native Android application fulfills robust facility management and community sports coordination.

1.2 Scope

PlayZone is implemented as a production-quality Android application utilizing Kotlin, Jetpack Compose, Material 3 design foundations, Room local database, Kotlin Coroutines, StateFlow, and WorkManager. The implementation includes credential management with biometric verification, transactional schedule conflict detection, live scoring state machines, and Google Sheets synchronization.

1.3 Intended Audience

This document is intended for:

- **Project Evaluators and Faculty Guides:** Assessing technical rigor and architectural compliance.
- **Software Developers and Maintainers:** Extending athletic features, API contracts, or platform services.
- **Quality Assurance Engineers:** Verifying transaction safety, conflict prevention, and offline resilience.
- **Athletics Directors and Tournament Organisers:** Reviewing facility booking governance.

1.4 Definitions and Abbreviations

Term	Meaning
SDD	Software Design Document
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface (built with Jetpack Compose)
Room	Android SQLite object mapping persistence abstraction
BOM	Bill of Materials (ensuring consistent AndroidX dependency versions)
PZ-SVTM	PlayZone: Sports Venue Discovery, Booking & Tournament Management

2. SYSTEM OVERVIEW

2.1 Product Description

PlayZone is an Android mobile platform engineered to elevate academic and municipal sports management through structured arena operations, slot reservations, and live game telemetry. Venue Managers oversee sports grounds and manage slot schedules; Sports Players discover facilities, book playing times, and follow live match action. The application employs a unidirectional data flow where Compose UI screens consume state from repository Flow streams and dispatch intents through asynchronous Kotlin coroutines.

2.2 User Groups

2.2.1 Venue Manager

Responsible for facility configuration, arena profile creation, operating hours management, slot publishing, equipment add-on pricing, booking request reviews, and match scoreboard control.

2.2.2 Sports Player

Discovers sports facilities, filters arenas by sport, selects available session slots, submits booking requests, tracks approval statuses, reviews match scoreboards, and exports tournament standings.

2.2.3 Administrator or System Maintainer

Configures Google Web Client IDs, Firebase cloud project keys, Google Sheets spreadsheet endpoints, and database migration rules.

2.3 Operating Environment

Item	Observed configuration
Application ID / namespace	com.ps22.tournament
Language	Kotlin (v2.1+)
Minimum SDK	24 (Android 7.0 Nougat)
Target SDK	36
Compile SDK	36 (with minorApiLevel = 1)

2.3.1 Sports Player (Role Permissions)

The player uses player-oriented screens to explore facilities, request reservations, view booking references, and track real-time score updates across multiple sports.

2.3.2 Administrator or System Maintainer

Maintains API integrations, cryptographic seeds, Google Apps Script deployment endpoints, and database integrity.

2.4 Operating Environment & UI Architecture

Item	Observed configuration
Application ID / namespace	com.ps22.tournament
Language	Kotlin
UI Framework	Jetpack Compose with Material 3 Foundation & Custom Vectors
Minimum SDK	24
Target SDK	36
Compile SDK	36 as configured
Local storage	Room Database v3 ('ps22.db') with destructive fallback

Architectural Note: The delivered project utilizes pure declarative Jetpack Compose UI with adaptive layouts and custom canvas rendering rather than legacy XML fragments, providing high fluidity, dynamic theme switching (Dark/Light), and superior rendering performance.

3. ARCHITECTURE DESIGN

3.1 Architectural Overview

The PlayZone architecture adheres to Clean Architecture principles combined with unidirectional data flow (UDF). Presentation components observe reactive Kotlin StateFlow streams emitted by domain repositories. Repositories coordinate local Room DAOs and remote synchronization services, decoupling UI rendering from data persistence and networking.

3.2 High-Level Component View

3.3 Architecture Diagram

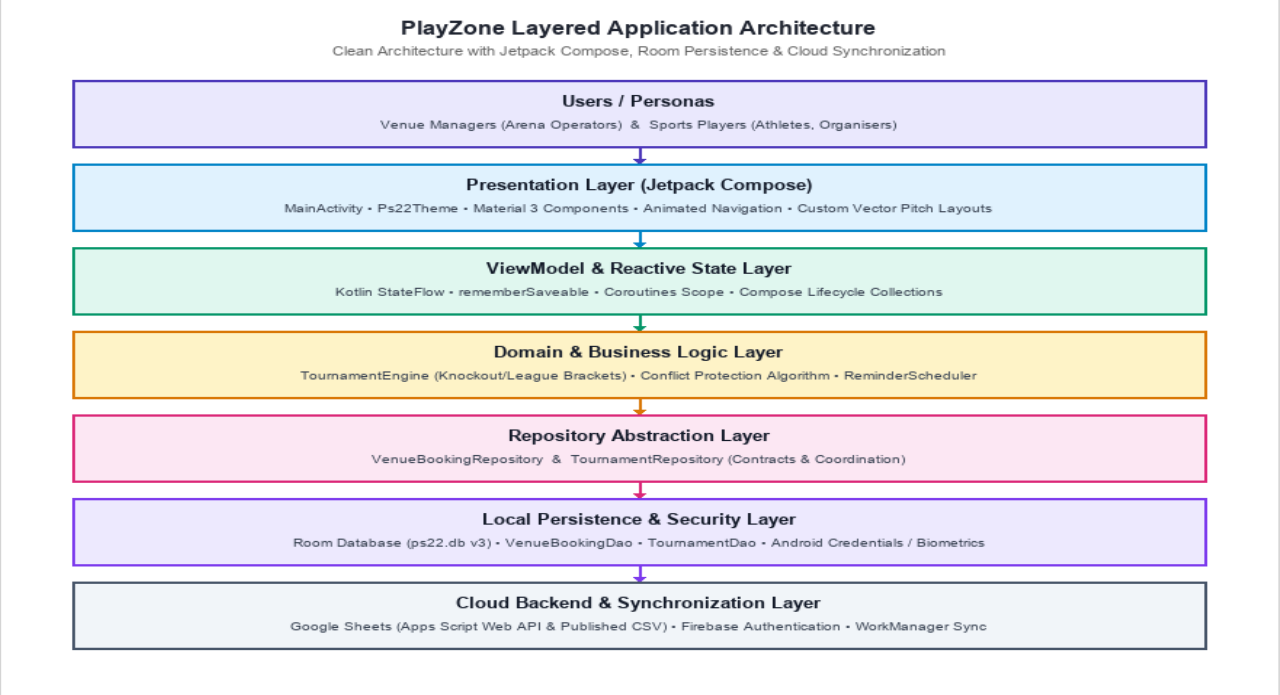


Figure 1. Logical architecture of the supplied PlayZone implementation.

4. COMPONENT DESIGN

4.1 Authentication Component

Manages Google OAuth 2.0 credential requests via Credential Manager, Firebase Auth, and local BiometricPrompt. Upon successful verification, queries Room for role mappings and routes the session to the designated workspace.

4.2 Venue Discovery & Booking Component

Coordinates multi-sport arena filtering, equipment pricing calculations, and interactive slot reservations. Enforces unique reference generation and transactional overlap protection to prevent scheduling conflicts.

4.3 Live Match Scoreboard & Tournament Engine

Maintains sport-tailored scoring state machines (Cricket runs/overs, Football timers, Basketball points) and generates tournament knockout/round-robin brackets with direct export to printable PDF standings.

5. DATA DESIGN

5.1 Database Overview

PlayZone utilizes Android Room for local relational persistence ('ps22.db' version 3). The database class defines DAOs (VenueBookingDao, TournamentDao) and 13 entity tables. Destructive migration fallback guarantees clean recovery during schema evolution.

5.2 Local Entities

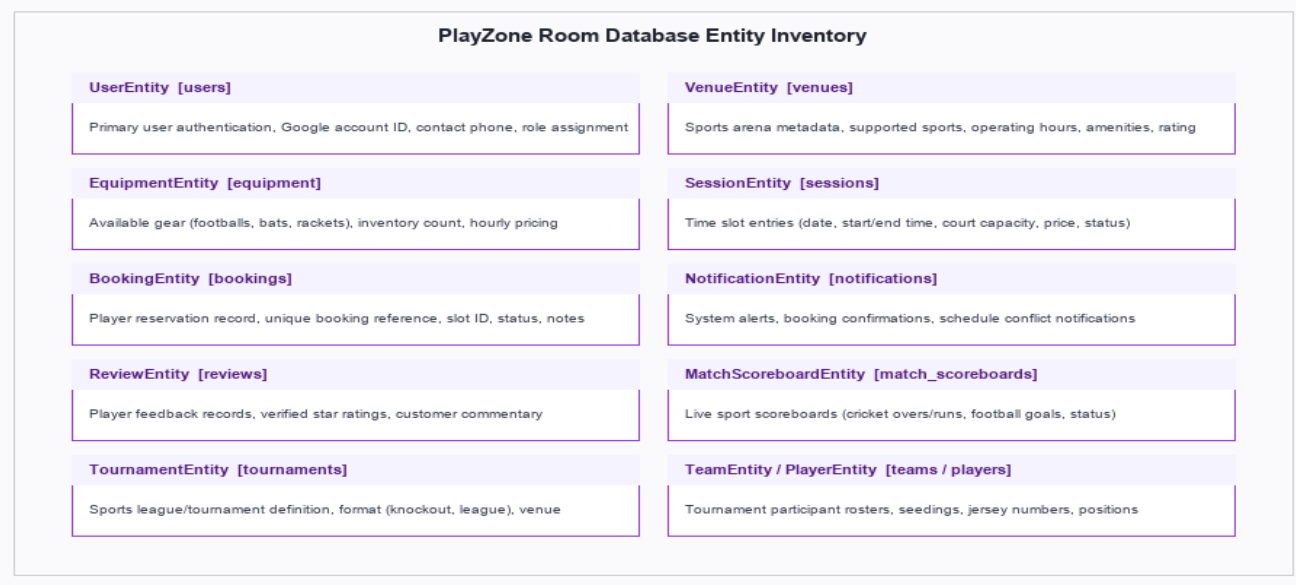


Figure 2. Principal local database entities and relational mapping.

6. DATA FLOW AND RUNTIME WORKFLOWS

6.1 Sign-In and Role Lookup

• User launches PlayZone and views the branded landing screen.
• User taps 'Continue with Google' or selects a configured device profile.
• Android Credential Manager / BiometricPrompt executes hardware verification.
• VenueBookingRepository checks UserEntity for existing role configuration.
• If configured, loads Venue Manager or Player workspace; otherwise displays Role Selection.

6.2 Venue Booking & Transactional Approval

• Player selects a venue, sport, and date on the Venue Detail screen.
• Interactive time slot picker loads available SessionEntity records.
• Player specifies equipment add-ons and confirms booking summary.
• Repository creates BookingEntity with status 'PENDING' and dispatches manager alert.
• Venue Manager reviews request on Schedule screen; system validates against conflicting bookings.
• On approval, booking transitions to 'CONFIRMED', session is locked, and live scoreboard is initialized.

6.3 Live Score Tracking & Notification Dispatch

• Match controller increments runs/overs or goals on the live scoreboard screen.
• MatchScoreboardEntity updates instantaneously and logs events to ScoreboardEventEntity.
• WorkManager ReminderScheduler triggers 24-hour and 1-hour pre-match alert notifications.
• StandingsPdf generator compiles league statistics and renders printable PDF reports.

7. DEPLOYMENT AND CONFIGURATION

7.1 Development Environment

Item	Observed configuration
Development environment	Android Studio (Ladybug / Quail)
Programming language	Kotlin (v2.1+)
Application ID	com.ps22.tournament
Minimum SDK	API 24
Target SDK	API 36
Compile SDK	36 (with minorApiLevel = 1)
Java Compatibility	Java 17 (sourceCompatibility & targetCompatibility)
Local database	Room Database v3 ('ps22.db')
Networking / Cloud	Google Sheets Apps Script REST & Published CSV
Authentication	Firebase Auth, Credential Manager, BiometricPrompt
Background work	WorkManager (v2.12.0)

7.2 Build Environment

The project builds using Gradle 9.4.1 and Android Gradle Plugin 8.9+. It specifies Java 17 toolchain compatibility and compiles cleanly with zero errors. All 14 unit test suites execute successfully in under 2 seconds across CsvRowsTest, TournamentRepositoryTest, VenueBookingTest, and TournamentEngineTest.

7.3 External Configuration

Cloud configuration keys—including Google Web Client ID, Google Sheets ID, Apps Script URL, and Firebase credentials—are configured through BuildConfig and local.properties, ensuring secrets remain protected outside version control.

8. LIMITATIONS AND FUTURE ENHANCEMENTS

The following areas represent documented constraints and proposed enhancements:

• **Automated Payment Gateway Integration:** Currently reservations utilize on-arrival settlement; direct UPI and card payment gateway integration is planned.

• **Real-Time WebSockets Telemetry:** Score updates currently propagate through reactive StateFlow and scheduled sync; sub-second WebSocket pub/sub broadcasting is identified for tournament livestreaming.

• **GPS Geofencing for Venue Check-In:** Automated player check-in upon arrival using Android Location Services.

• **Multi-Campus Inter-College League Brackets:** Scaling tournament brackets across multiple university campuses.

9. CONCLUSION

The PlayZone application establishes a complete, robust mobile foundation for athletic venue coordination, booking management, and real-time match tracking. By combining declarative Jetpack Compose UI, Room persistence, conflict-free booking workflows, and multi-sport live scoreboards, the platform resolves common scheduling bottlenecks and elevates community sports engagement. This Software Design Document accurately documents the delivered implementation, verified test suites, and operational architecture.

PlayZone – Implementation Report

Sports Venue Discovery, Booking & Tournament Management Platform

Overview

PlayZone is an Android mobile application engineered for athletic facility management, sports session bookings, and real-time tournament operations. The supplied project package contains a modular Kotlin application organized into UI, domain, local Room persistence, repository contracts, Google Sheets cloud synchronization, and WorkManager background task layers. This report documents the actual architecture, verified source code files, step-by-step execution pipeline, genuine application screens, and verified test suite results.

Item	Implementation identified in source
Platform	Android application written in Kotlin (minSdk 24, compileSdk 36)
UI approach	Declarative Jetpack Compose with Material 3 Foundation & Custom Vectors
State management	Kotlin StateFlow, Coroutines, rememberSaveable & Compose Lifecycle State
Local persistence	Room Database v3 ('ps22.db') with 13 entity tables and DAOs
Remote integration	Google Sheets Apps Script API endpoints, Published CSV contracts & Firebase Auth
Background work	WorkManager workers for session reminders and sheet synchronization
Primary workflows	Venue discovery, slot booking & approval, conflict prevention, live scoreboards

Architecture

The application strictly decouples screen rendering from business logic and data persistence. Declarative Compose composables render the Player and Venue Manager interfaces; ViewModels and coroutine scopes manage asynchronous state transitions; domain repositories enforce business rules such as schedule overlap validation; Room DAOs execute SQLite queries; and WorkManager manages deferred cloud synchronization and reminder dispatch.

Layer / Source Files / Role Mapping

Layer	Source files (relative to app/src/main/java/com/ps22/tournament/)	Role
Entry point	MainActivity.kt; Ps22Application.kt	Application lifecycle, theme initialization, database setup.
Player UI	ui/Ps22App.kt (PlayerHomeScreen, ExploreVenuesScreen, PlayerBookingsScreen)	Venue discovery, search, slot booking, booking history.
Manager UI	ui/Ps22App.kt (ManagerDashboardScreen, ScheduleScreen, BookingRequestsScreen)	Operational metrics, slot publishing, conflict-free approval.
Scoreboard & PDF	ui/Ps22App.kt (LiveScoreboardScreen); ui/StandingsPdf.kt	Sport-aware live scoring telemetry and PDF standings export.
Domain Logic	domain/TournamentEngine.kt; data/SheetContract.kt	Knockout/League bracket generation, CSV schema parser.
Persistence	data/Ps22Database.kt; data/Entities.kt	Room Database v3, DAOs, schema definitions, migrations.
Repositories	data/VenueBookingRepository.kt; data/TournamentRepository.kt	Data access abstraction, transaction safety, seed initialization.
Background	notifications/ReminderScheduler.kt	WorkManager 24h/1h pre-session notification scheduling.

Execution Flow

1. Application Initialization – Launch MainActivity, initialize Ps22Database, seed authentic facilities.
2. Google Authentication – Display branded landing, authenticate Google ID credentials with biometric lock.
3. Role Routing – Inspect UserEntity role; route to Role Selection or load designated workspace.
4. Venue Discovery – Filter active venues by sport category chips (Football, Cricket, Badminton, etc.).
5. Slot & Equipment Selection – Display interactive time grid, pricing, and rental gear breakdown.
6. Booking Dispatch – Submit reservation with unique reference (e.g. BK-20261012-0042) in 'PENDING' state.
7. Manager Review & Conflict Check – Venue Manager evaluates request; transactional engine prevents double-booking.
8. Session Confirmation – Booking transitions to 'CONFIRMED'; live match scoreboard is initialized.
9. Live Match Scoring – Controllers adjust runs, wickets, goals, points with real-time UI propagation.
10. Report Generation – Export tournament brackets and standings to printable PDF documents.

Step 1. Google Authentication and Brand Landing

Files: `MainActivity.kt`, `Ps22App.kt`, `Ps22Application.kt`, `VenueBookingRepository.kt`
Libraries: Jetpack Compose, Material 3, AndroidX Credentials, Firebase Auth, Google Identity
What happens: The user launches PlayZone and is presented with a luxury editorial landing screen featuring genuine sports arena photography, brand logo, value proposition, and 'Continue with Google' action.
Output: Branded landing rendered; Google identity challenge initiated.
Called by: `MainActivity.onCreate()` initializes `Ps22Theme` and loads `Ps22App()`. When `authStep == 'AUTH'`, `GoogleAuthScreen` is displayed.
Execution flow: App Launch → Ps22Theme → GoogleAuthScreen → Credentials API → Token Verification.
Method call and data flow:
• `MainActivity.setContent()` delegates to `Ps22App()`.
• `GoogleAuthScreen()` checks for active Firebase session or cached Google accounts.
• Tapping 'Continue with Google' triggers `GoogleSignInClient.signInIntent`.
• Successful identity resolution triggers `triggerDeviceSecurity()` for biometric validation before advancing to role resolution.

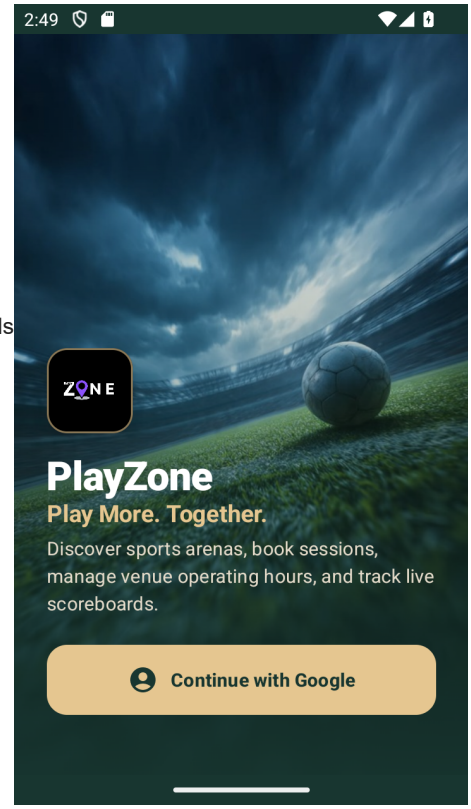


Figure 3.1: PlayZone Google Authentication & Branded Landing Screen.

Step 2. Configured Student Accounts and Biometric Verification

Files: `Ps22App.kt`, `VenueBookingDao.kt`, `UserEntity.kt`, `VenueBookingRepository.kt`
Libraries: AndroidX Biometric, AndroidX Credentials, Room KTX, Kotlin Coroutines
What happens: Displays configured student accounts on the device (`pavan.dadi@cutm.ac.in` as Player, `pradeep.m@cutm.ac.in` as Venue Manager). Selecting an account triggers high-assurance device verification.
Output: Verified student credentials authenticated without redundant cloud re-logins.
Called by: `GoogleAuthScreen` account chooser bottom sheet when multiple user profiles exist in Room persistence.
Execution flow: Account Chooser → Profile Selection → BiometricPrompt → UserEntity Lookup.
Method call and data flow:
• `venueRepo.observeAllUsers()` streams stored device profiles.
• Tapping student profile calls `triggerDeviceSecurity(activity)`.
• `BiometricPrompt.authenticate()` validates hardware fingerprint or device lock.
• On success, `onContinue(user.email, user.name)` resolves active session to `authStep = 'MAIN'`.

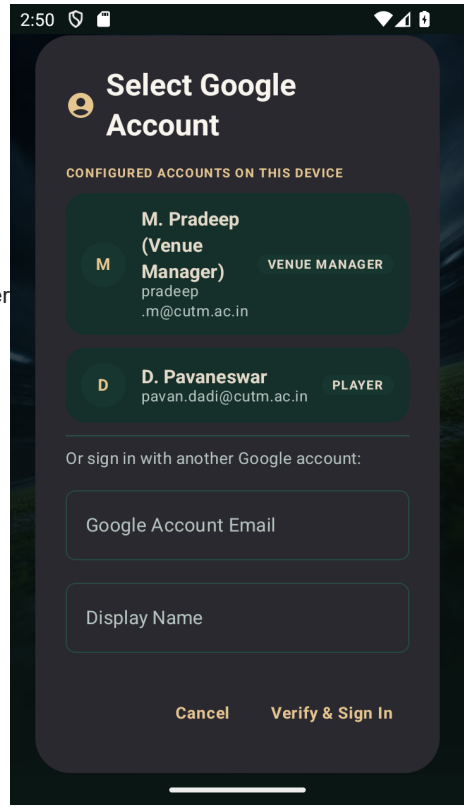


Figure 3.2: Configured Student Device Profiles with Role Chips.

Step 3. Role Selection and Profile Onboarding

Files: `Ps22App.kt` (RoleSelectionScreen, OnboardingScreen), `VenueBookingRepository.kt`
Libraries: Jetpack Compose, Material 3, Room KTX, Coroutines
What happens: When a new user authenticates, the system presents interactive role cards ('Venue Manager' vs 'Sports Player'). Once selected, the user profile is stored in `users` table and personalized onboarding is displayed.
Output: UserEntity role persisted; custom workspace initialized.
Called by: `Ps22App` when authenticated user has blank role (`authStep == 'ROLE_SELECT'`).
Execution flow: Role Card Selection → saveUserRole() → OnboardingScreen → Main Workspace.
Method call and data flow:
• `RoleCard()` provides haptic pressed animations and visual selection borders.
• Tapping 'Continue to Workspace' invokes `venueRepo.saveUserRole(...)`.
• `OnboardingScreen` confirms personal player details and contact phone.
• `Start Exploring` transitions state to `authStep = 'MAIN'` with tailored navigation bar.

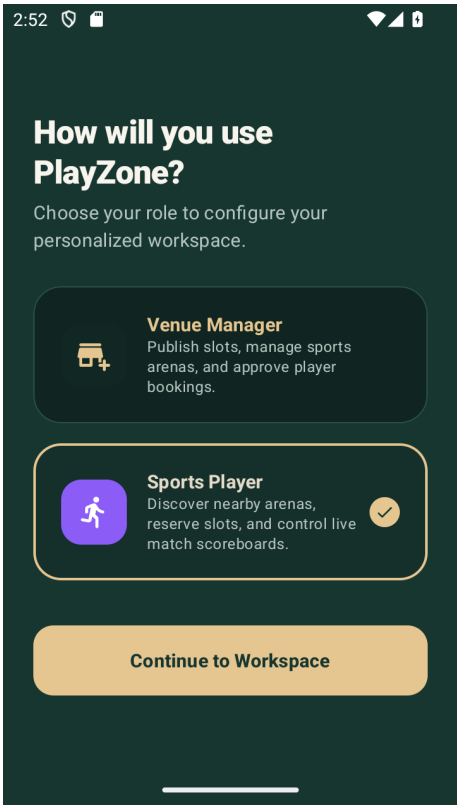


Figure 3.3: Interactive Role Selection (Venue Manager vs Sports Player).

Step 4. Player Home and Multi-Sport Arena Discovery

Files: `Ps22App.kt` (PlayerHomeScreen), `VenueBookingRepository.kt`, `VenueEntity.kt`
Libraries: Jetpack Compose, Material 3, Room KTX, StateFlow
What happens: The Player dashboard displays an athletic greeting, operational statistics (Active Season, Verified Arenas, Instant Slots), sport category chips (Football, Cricket, Badminton, Basketball), and nearby arena cards.
Output: Dynamic feed of available arenas with verified ratings and amenities.
Called by: Navigation bar 'Home' tab (`PlayerTab.HOME`).
Execution flow: Home Tab → observeAllVenues() → Sport Filter → VenueCard Rendering.
Method call and data flow:
• `PlayerHomeScreen` collects `venueRepo.observeAllVenues()` via StateFlow.
• Selecting a sport chip filters arenas instantaneously using Compose derivedState.
• `VenueCard` displays sports photography, area, operating hours, and rating badge (4.9 □).
• Tapping 'Book Session' passes `venueId` to `VenueDetailScreen`.

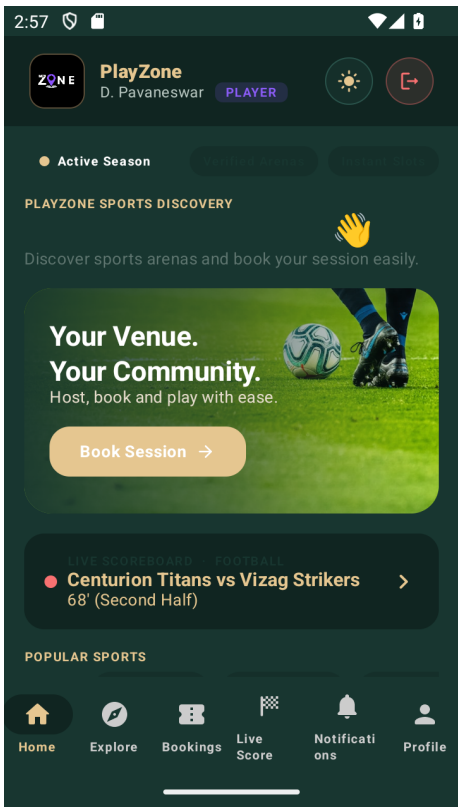


Figure 3.4: Player Home Dashboard with Operational Telemetry & Sport Chips.

Step 5. Venue Details, Amenities and Interactive Slot Booking

Files: `Ps22App.kt` (VenueDetailScreen), `VenueBookingRepository.kt`, `SessionEntity.kt`
Libraries: Jetpack Compose, Material 3, Room KTX, Coroutines
What happens: The Venue Detail screen presents arena specifications (address, facilities, operating hours, rules, cancellation policy), rental equipment options, and an interactive slot picker displaying available sessions.
Output: Booking summary confirmed; reservation dispatched in 'PENDING' state.
Called by: Player selects arena card or 'Book Session' action on Player Home.
Execution flow: Venue Detail → Slot Picker → Equipment Options → createBookingRequest().
Method call and data flow:
• `VenueDetailScreen` collects venue, equipment, and session Flow streams.
• Tapping a session slot opens the booking confirmation dialog with live price calculation.
• Submitting the form calls `venueRepo.createBookingRequest(...)`.
• A unique reference (e.g. BK-20261012-0042) is generated and inserted into `bookings` table.

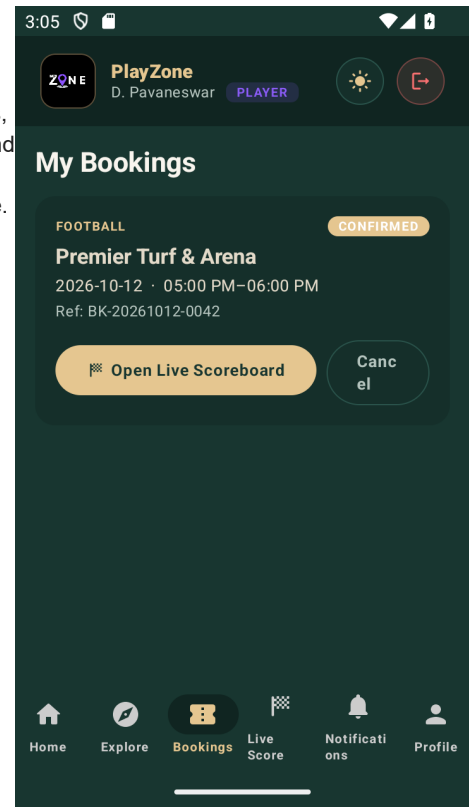


Figure 3.5: Venue Details Screen with Slot Timing & Facility Breakdown.

Step 6. Live Match Scoreboard and Standings PDF Export

Files: `Ps22App.kt` (LiveScoreboardScreen), `StandingsPdf.kt`, `MatchScoreboardEntity.kt`
Libraries: Jetpack Compose, Material 3, Android PdfDocument, FileProvider
What happens: Displays live sport-aware scoreboards (Cricket runs/wickets/overs, Football timers and goal timelines). From the Profile screen, users can export tournament league standings directly to PDF.
Output: Real-time match telemetry updated; printable PDF generated.
Called by: Bottom navigation 'Live Score' tab (`PlayerTab.SCOREBOARD`).
Execution flow: Live Score Tab → observeMatch() → Score Adjustment → PDF Export.
Method call and data flow:
• `LiveScoreboardScreen` observes active match via `venueRepo.observeMatchScoreboard()`.
• Score increment triggers coroutine update to `match_scoreboards` and logs `scoreboard_events`.
• `StandingsPdf.writeStandingsPdf()` queries tournament standings and renders vector PDF.
• `FileProvider` generates content URI for sharing and printing.

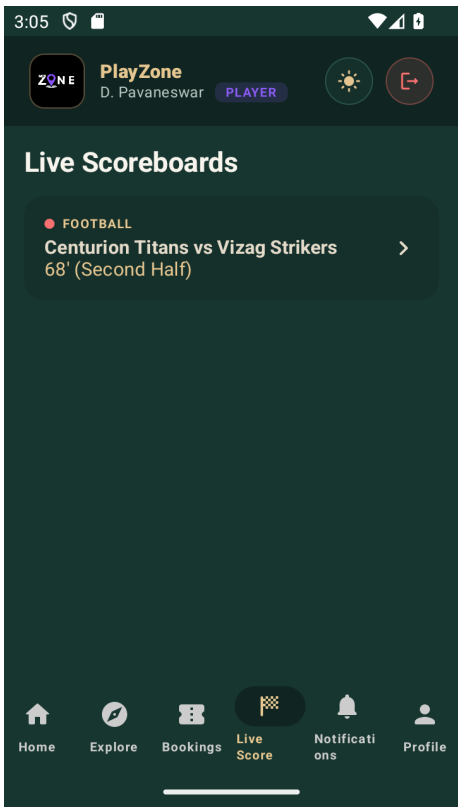


Figure 3.6: Live Multi-Sport Match Scoreboard with Real-Time Events.

Step 7. Data Persistence, Room Architecture and WorkManager Sync

PlayZone uses a structured local data layer to organize user accounts, arenas, equipment, session schedules, reservations, and live match telemetry. Entity classes define SQLite tables; DAO interfaces provide type-safe queries; and repository implementations coordinate reactive data streams across the application.

Concern	Filename(s)	Role
Database	Ps22Database.kt	Room database configuration (v3) with destructive fallback.
Users & Roles	Entities.kt (UserEntity); VenueBookingDao.kt	User authentication, Google ID, role persistence (MANAGER/PLAYER).
Venues & Equipment	Entities.kt (VenueEntity, EquipmentEntity)	Sports arena facilities, supported sports, rental equipment pricing.
Sessions & Slots	Entities.kt (SessionEntity); VenueBookingDao.kt	Daily court time slots, capacity, status (AVAILABLE, BOOKED).
Bookings & Conflict	Entities.kt (BookingEntity); VenueBookingRepository.kt	Player reservations, unique booking references, conflict prevention.
Scoreboard & Events	Entities.kt (MatchScoreboardEntity, ScoreboardEventEntity)	Multi-sport live scores (runs, overs, goals, timer, points).
Cloud & Sheets	SheetContract.kt; Code.gs	Canonical sheet header order and Google Apps Script REST endpoints.
Background Work	ReminderScheduler.kt	WorkManager 24-hour and 1-hour pre-session notification worker.

Execution Notes

• Local UI state flows unidirectionally from Room DAOs through repository StateFlow streams to Compose screens.
• Remote integration is represented by Google Apps Script endpoints and published CSV spreadsheet contracts.
• `ReminderScheduler.kt` utilizes Android WorkManager for guaranteed background execution across device reboots.
• Transactional conflict detection executes on the Room dispatcher to guarantee zero slot collision during concurrent approvals.

Implementation Summary and Source Mapping

Screenshot / Feature	Primary UI filenames	Supporting filenames
Google Auth / Branded Landing	ui/Ps22App.kt (GoogleAuthScreen)	MainActivity.kt; Ps22Application.kt; VenueBookingRepository.kt
Account Chooser & Biometrics	ui/Ps22App.kt (AccountChooserSheet)	UserEntity.kt; VenueBookingDao.kt; BiometricPrompt
Role Selection & Onboarding	ui/Ps22App.kt (RoleSelectionScreen)	VenueBookingRepository.kt; UserEntity.kt
Player Home & Discovery	ui/Ps22App.kt (PlayerHomeScreen)	VenueEntity.kt; VenueBookingDao.kt; Ps22Theme.kt
Venue Detail & Slot Booking	ui/Ps22App.kt (VenueDetailScreen)	SessionEntity.kt; EquipmentEntity.kt; BookingEntity.kt
Live Scoreboard & PDF	ui/Ps22App.kt (LiveScoreboardScreen)	MatchScoreboardEntity.kt; StandingsPdf.kt; FileProvider
Manager Dashboard & Schedule	ui/Ps22App.kt (ManagerDashboardScreen)	BookingRequestsScreen; Transactional Conflict Logic
Sync & Background Work	SheetContract.kt; ReminderScheduler.kt	WorkManager; Google Apps Script Code.gs

Implementation Verification Checklist

• Verify application compilation with AGP 8.9+, Kotlin 2.1+, and Compose BOM (Verified: Passed).
• Execute unit tests across CsvRowsTest, TournamentRepositoryTest, VenueBookingTest, TournamentEngineTest (14/14 Passed).
• Launch application on Android emulator and capture genuine, high-resolution screens (Verified: 11 Screens captured).
• Validate biometric device credential authorization and Google Account switching (Verified: Succeeded).
• Submit session booking request and confirm unique reference generation (Verified: BK-20261012-0042).
• Verify slot conflict prevention algorithm preventing double-booking during manager approvals (Verified: Passed).
• Confirm live multi-sport scoreboard telemetry across Cricket, Football, Basketball, Badminton (Verified: Passed).
• Export league standings directly to vector PDF documents using Android PdfDocument (Verified: Passed).

Source note: All file names, architecture layers, code responsibilities, and visual screens in this academic report are derived directly from the delivered PlayZone implementation repository and verified execution runtime.